

Matrix Class

Class Design Scratch

We need to first decide what functionalities our matrix class should have. According to the project guidelines, it should be able to:

- handle different types of elements, including unsigned char, short, int, float, double, etc.
- implement region of interest (ROI) to avoid memory hard copy
- overload operators including but not limited to `=`, `==`, `+`, `-`, `*`, etc.
- reuse `project3` if possible

As a heavy `MATLAB` user and a `CUDA` beginner, I wish to :

- use the matrix in a `MATLAB` -like way
- write the code with `CUDA`

Now, we can further break these requirements into more detailed instructions:

- Implement the class with a template so that it can handle different types
- Avoid hard copy when overloading `operator=`
- Implement ROI with `operator()`
- Use `CUDA` in the code

Class Attributes

Since we are using a 1-D array to store our matrix, it would be troublesome when implementing ROI because the regional matrix is not stored in a continuous manner. As a result, we need to store the size of the original matrix in our regional matrix as well. Thus, four values are introduced: `m_offset`, `n_offset`, `m_pad` and `n_pad`. The first two values represent the offset of the first element of the sub-matrix, and the other two values represent the height and weight of the original matrix.

Furthermore, since matrices can share the same chunk of an array by using a pointer, to avoid double free and memory leak problems, a `std::shared_ptr<T>` called is introduced.

```

template <typename T>
class Matrix {
public:
    size_t m;           // Number of rows
    size_t n;           // Number of columns
    size_t m_offset;    // Offset for submatrix
    size_t n_offset;    // Offset for submatrix
    size_t m_pad;       // Number of rows padded to a multiple of BLOCK_SIZE
    size_t n_pad;       // Number of columns padded to a multiple of BLOCK_SIZE
    std::shared_ptr<T> array; // Pointer to the matrix elements
}

```

Create Matrix

The matrix can be initialized by two means:

```

Matrix<float> A(8, 8);
float array[9] = {1.0f, 2.0f, 3.0f, 4.0f, 5.0f, 6.0f, 7.0f, 8.0f, 9.0f};
Matrix B(3, 3, array);

```

Operator Overloading

Assigning Operator

= is overloaded for soft copy and assigning purposes:

- For regular `Matrix& operator=(const Matrix& other)` , our class will not copy the value of the right matrix but reassign the pointer towards its array.
- By overloading `Matrix& operator=(const T& value)` , `A = 1.5f;` can assign the whole matrix with the same value.
- One can also assign the values of the matrix using the following grammar:

```
c++    D = {1, 2, 3, 4, 5, 6, 7, 8, 9};
```

this is done by overloading `Matrix& operator=(std::initializer_list<T> list)` .

Region of Interest

The design of ROI is done by overloading

`operator()(size_t m_start, size_t n_start, size_t m_end, size_t n_end)` , representing

where the sub-matrix starts and ends. Using this operator, we can define the sub-matrix of an existing matrix like:

```
Matrix<float> C = A(3,3,6,6);
```

This sub-matrix can participate in all sorts of arithmetic operations. However, since `operator=` is assigned as soft copy, to change the values of the sub-matrix, we need to use member function `void copy(const Matrix& other) :`

```
void copy(const Matrix& other) {
    assert(this->m == other.m && this->n == other.n);
    # pragma omp parallel for
    for (size_t j = 0; j < this->n; j++) {
        for (size_t i = 0; i < this->m; i++) {
            (*this)(i,j) = other(i,j);
        }
    }
}
```

here `other(i,j)` and `(*this)(i,j)` are overloaded as

```
const T& operator()(size_t m_index, size_t n_index) const {
    assert(m_index < m && n_index < n);
    return array.get()[(m_index + m_offset) + (n_index + n_offset) * m_pad];
}

T& operator()(size_t m_index, size_t n_index) {
    assert(m_index < m && n_index < n);
    return array.get()[(m_index + m_offset) + (n_index + n_offset) * m_pad];
}
```

To simplify coding. We need two separate overloading sessions here for both reading and writing.

Cuda

In the previous project, `SIMD` and `OPENMP` were used to boost matrix multiplication, `OPENMP` is often a no-brainer optimization and, therefore, is still used in the project; however, since we need our template to deal with different data types, and `SIMD` is too hardware-wised, using `SIMD` means that we have to write many different cases, which violates our initial purpose of using template.

Fortunately, we can still use Cuda to boost our matrix operations.

Since using cuda will involve copying arrays from and to GPU, we can first define an operator wrapper:

```
template <typename T, typename F>
void matrixOpWrapper(Matrix<T>& C, const Matrix<T>& A, const Matrix<T>& B, F f){
    T *d_A, *d_B, *d_C;
    size_t size_A = A.m * A.n * sizeof(T);
    size_t size_B = B.m * B.n * sizeof(T);
    size_t size_C = C.m * C.n * sizeof(T);

    // Allocate memory on the GPU
    size_t pitch_A, pitch_B, pitch_C;

    cudaMallocPitch((void**)&d_A, &pitch_A, A.m * sizeof(T), A.n);
    cudaMallocPitch((void**)&d_B, &pitch_B, B.m * sizeof(T), B.n);
    cudaMallocPitch((void**)&d_C, &pitch_C, C.m * sizeof(T), C.n);

    // Copy matrices from the host to the device
    cudaMemcpy2D(d_A, A.m * sizeof(T), A.array.get() + A.m_offset + A.n_offset * A.m_pad, A
    cudaMemcpy2D(d_B, B.m * sizeof(T), B.array.get() + B.m_offset + B.n_offset * B.m_pad, B

    // Define the dimensions of the grid and blocks
    dim3 threadsPerBlock(16, 16);
    dim3 numBlocks((B.n + threadsPerBlock.x - 1) / threadsPerBlock.x, (A.m + threadsPerBlock

    // Call the matrix operation function
    f<<<numBlocks, threadsPerBlock>>>(d_C, d_A, d_B, A.m, A.n, B.n);

    // Copy the result matrix from the device to the host
    cudaMemcpy2D(C.array.get() + C.m_offset + C.n_offset * C.m_pad, C.m_pad * sizeof(T), d_C
    // Free the memory allocated on the GPU
    cudaFree(d_A);
    cudaFree(d_B);
    cudaFree(d_C);
}
```

Here we use `cudaMallocPitch` and `cudaMemcpy2D` rather than `cudaMalloc` and `cudaMemcpy`, because we need to copy sub-matrices whose values are not stored continuously in the memory. According to [this blog](#), we can use `spitch` to input the pitch of source memory and let the cuda know when to jump to the next columns.

With this operator wrapper, we can define the add, subtract and multiply of cuda as followed:

```
template <typename T>
__global__ void matrixMulCUDA(T *C, T *A, T *B, int m, int n, int k){
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if(row < m && col < k) {
        T sum = 0;
        for(int i = 0; i < n; i++) {
            sum += B[row * n + i] * A[i * k + col];
        }
        C[row * k + col] = sum;
    }
}

template <typename T>
__global__ void matrixAddCUDA(T *C, T *A, T *B, int m, int n, int k){
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if(row < m && col < n) {
        C[row * n + col] = A[row * n + col] + B[row * n + col];
    }
}

template <typename T>
__global__ void matrixSubCUDA(T *C, T *A, T *B, int m, int n, int k){
    int row = blockIdx.y * blockDim.y + threadIdx.y;
    int col = blockIdx.x * blockDim.x + threadIdx.x;

    if(row < m && col < n) {
        C[row * n + col] = A[row * n + col] - B[row * n + col];
    }
}
```

Now we have got the function templates for basic arithmetic operations, we can overload them to the operator.

Arithmetic Operator

In the project, `+`, `-`, `*`, `+=`, `-=`, `==` are overloaded so that arithmetic operations can be done between matrices directly.

```

// Matrix multiplication
Matrix<T> operator*(const Matrix<T>& B) {
    assert(n == B.m); // Ensure the matrices can be multiplied
    Matrix<T> C(m, B.n);
    matrixOpWrapper(C, *this, B, matrixMulCUDA<T>);
    return C;
}

// Matrix addition
Matrix<T> operator+(const Matrix<T>& B) {
    assert(m == B.m && n == B.n); // Ensure the matrices have the same dimensions
    Matrix<T> C(m, n);
    matrixOpWrapper(C, *this, B, matrixAddCUDA<T>);
    return C;
}

//matrix self-addition
Matrix<T>& operator+=(const Matrix<T>& B) {
    assert(m == B.m && n == B.n); // Ensure the matrices have the same dimensions
    matrixOpWrapper(*this, *this, B, matrixAddCUDA<T>);
    return *this;
}

//matrix subtraction
Matrix<T> operator-(const Matrix<T>& B) {
    assert(m == B.m && n == B.n); // Ensure the matrices have the same dimensions
    Matrix<T> C(m, n);
    matrixOpWrapper(C, *this, B, matrixSubCUDA<T>);
    return C;
}

//matrix self-subtraction
Matrix<T>& operator-=(const Matrix<T>& B) {
    assert(m == B.m && n == B.n); // Ensure the matrices have the same dimensions
    matrixOpWrapper(*this, *this, B, matrixSubCUDA<T>);
    return *this;
}

// Matrix equality
bool operator==(const Matrix<T>& B) {
    if (m != B.m || n != B.n) {
        return false;
    }
}

```

```

    if (this->array == B.array) {
        return true;
    }
    for (size_t i = 0; i < m; i++) {
        for (size_t j = 0; j < n; j++) {
            if ((*this)(i,j) != B(i,j)) {
                return false;
            }
        }
    }
    return true;
}

```

IO Operator

To input and output matrices from the command line, we also overloaded

friend std::ostream& operator<<(std::ostream& os, const Matrix<T>& matrix) and
Matrix& operator>>(std::istream& is) so that the matrix can be print and input using cin and
cout .

```

// Get matrix from cin
Matrix& operator>>(std::istream& is) {
    std::cout << "Enter the matrix size (m n): ";
    is >> m >> n;
    this->m = m;
    this->n = n;
    this->m_offset = 0;
    this->n_offset = 0;
    this->m_pad = m;
    this->n_pad = n;
    this->array = std::shared_ptr<T>((T*)aligned_alloc(64, this->m_pad * this->n_pad * sizeof(T)));
    for (size_t j = 0; j < n; j++) {
        for (size_t i = 0; i < m; i++) {
            is >> (*this)(i,j);
        }
    }
    return *this;
}

```

```
// Print the matrix
friend std::ostream& operator<<(std::ostream& os, const Matrix<T>& matrix) {
    for (size_t j = 0; j < matrix.m; j++) {
        for (size_t i = 0; i < matrix.n; i++) {
            printElement(matrix(i,j));
        }
        os << "\n";
    }
    return os;
}
```

Test Result

With the class defined above, we can now test if the class is correctly defined:

```

#include <iostream>
#include "sharedMatrix.cu"
#include <sys/time.h>

int main() {
// Test the default constructor
Matrix<float> A(8, 8);

// Test the constructor with an existing array
float array[9] = {1.0f, 2.0f, 3.0f, 4.0f, 5.0f, 6.0f, 7.0f, 8.0f, 9.0f};
Matrix B(3, 3, array);
std::cout << "B:\n" << B << "\n";

// Test the assignment operator with a single value
A = 1.5f;
std::cout << "A:\n" << A << "\n";

Matrix<float> C = A(3,3,6,6);
C.copy(B*B);
A(1,1,4,4) += C;
cudaError_t err = cudaGetLastError();
if (err != cudaSuccess) {
    std::cerr << "CUDA error: " << cudaGetErrorString(err) << std::endl;
}
std::cout << "C:\n" << C << "\n";
std::cout << "A:\n" << A << "\n";
std::cout << "A(3,3):\n" << A(3,3) << "\n";

Matrix<unsigned char> D(3, 3);
D = {1, 2, 3, 4, 5, 6, 7, 8, 9};
std::cout << "D:\n" << D << "\n";
D(1,1,3,3) = 0;
std::cout << "D:\n" << D << "\n";
D += D;
std::cout << "D:\n" << D << "\n";

Matrix<double> E(8000, 8000);
E = 1.0;
Matrix<double> F(8000, 8000);
F = 2.0;

struct timeval start, end;
gettimeofday(&start, NULL);

```

```

E = E*F;
gettimeofday(&end, NULL);
std::cout << "Time taken: " << (end.tv_sec - start.tv_sec) + (end.tv_usec - start.tv_usec) / 1000000.0 << "\n";

return 0;
}

```

And the output is as followed:

B:

1	2	3
4	5	6
7	8	9

A:

1.5	1.5	1.5	1.5	1.5	1.5	1.5	1.5
1.5	1.5	1.5	1.5	1.5	1.5	1.5	1.5
1.5	1.5	1.5	1.5	1.5	1.5	1.5	1.5
1.5	1.5	1.5	1.5	1.5	1.5	1.5	1.5
1.5	1.5	1.5	1.5	1.5	1.5	1.5	1.5
1.5	1.5	1.5	1.5	1.5	1.5	1.5	1.5
1.5	1.5	1.5	1.5	1.5	1.5	1.5	1.5
1.5	1.5	1.5	1.5	1.5	1.5	1.5	1.5

C:

180	36	42
66	81	96
102	126	150

A:

1.5	1.5	1.5	1.5	1.5	1.5	1.5	1.5
1.5	31.5	37.5	43.5	1.5	1.5	1.5	1.5
1.5	67.5	82.5	97.5	1.5	1.5	1.5	1.5
1.5	103.5	127.5	180	36	42	1.5	1.5
1.5	1.5	1.5	66	81	96	1.5	1.5
1.5	1.5	1.5	102	126	150	1.5	1.5
1.5	1.5	1.5	1.5	1.5	1.5	1.5	1.5
1.5	1.5	1.5	1.5	1.5	1.5	1.5	1.5

A(3,3):

180

```
D:
1      2      3
4      5      6
7      8      9
```

```
D:
1      2      3
4      0      0
7      0      0
```

```
D:
2      4      6
8      0      0
14     0      0
```

Time taken: 4.52883s

We can see that sub-matrices work perfectly with different types of data and operations.

Future Work

Stride

In MATLAB , it is often convenient to use two arrays to get a stride of the matrix like:

```
B = A({1,3,5}, {1,3,5});
```

This can be done by overloading the

```
operator()(std::initializer_list<T> row_list, std::initializer_list<T> col_list) ;
```

however, this would require adding more attributes to the matrix class itself; for the simplicity of the code, we will not do it here.

CUDA Optimization

Using RTX4060 as the testing bench, a matrix multiplication of size 8×10^3 takes 4.5 seconds, which is not very optimal for GPU work. In project 5, we will attempt to further optimize the parallelization level using blocking and shared cache like the work of [others](#).